SynthDef : Object

Client-side representation of a synth definition

Source: SynthDef.sc
Subclasses: ProxySynthDef, SynthDefOld

See also: Synth, Synth Definition File Format, SynthDesc

Description

The server application uses synth definitions as templates for creating Synth nodes. (Methods such as Function: -play, etc. are simply conveniences which automatically create such a def.) The SynthDef class encapsulates the client-side representation of a given def, and provides methods for creating new defs, writing them to disk, and streaming them to a server.

SynthDef is one of the more complicated classes in SC and an exhaustive explanation of it is beyond the scope of this document. As such, the examples at the bottom of this document and those found in the various tutorials accessible from Help may be necessary to make some aspects of its use clear.

Some insights about common SynthDef issues can be found on the FAQ SynthDef Issues

UGen Graph Functions and Special Argument Forms

The core of a def is its unit generator graph function. This is an instance of Function which details how the def's unit generators are interconnected, its inputs and outputs, and what parameters are available for external control. In a synth based on the def, arguments to the function will become instances of Control. These can have default values, or can be set at the time the synth is created. After creation they will be controllable through Node's `set` and `setn` methods, or the n_set and n_setn OSC messages.

There are four special types of arguments, which are treated differently:

audio rate

Arguments that begin with "a_" (e.g. `a_input`), or that are specified as `\ar` in the def's rates argument (see below), will be able to read an audio rate bus when mapped to it with `/n_mapa`.

initial rate

Arguments that begin with "i_" (e.g. `i_freq`), or that are specified as `\ir` in the def's rates argument (see below), will be static and non-modulatable. They will not respond to `/n_set` or `/n_map`. This is slightly more efficient in terms of CPU than a regular arg.

trigger rate

Arguments that begin with "t_" (e.g. `t_trig`), or that are specified as `\tr` in the def's rates argument (see below), will be made as a TrigControl. Setting the argument will create a control-rate impulse at the set value. This is useful for triggers.

literal arrays

Arguments which have literal arrays as default values (see Literals) result in multichannel controls, which can be set as a group with Node: -setn or `/n_setn`.

See the examples below for more detail on how this works.

Certain argument names (such as 'out' to specify an out bus) are in such common use that adopting them might be said to constitute 'good style'. One of these, 'gate' when used to control the gate input of an EnvGen, deserves special mention, as it allows one to use Node's release method. See Node for an example and more detail.

Arrayed controls

In some cases, it is useful to send a group of values to a single control name, for instance, `aSynth.set(\freqs, [300, 400, 500])`, or for passing envelopes at Synth instantiation. This requires an *arrayed control* to be defined in the SynthDef.

There are two ways to define arrayed controls:

1. NamedControl -- recommended for flexibility and clarity.

NamedControl can accept default values determined in code. For example, passing an envelope requires four values per breakpoint. It is easy to create a template for the arrayed control by writing `NamedControl.kr(\env, Env.newClear(numSegments: 8).asArray)`.

Literal arrays may be more convenient for very small arrays where all of the default values are known in advance.

In general, it is likely to be simpler in most cases to use NamedControl for arrayed controls.

See the example under Array Arguments.

Static versus Dynamic Elements

It is important to understand that although a single def can provide a great deal of flexibility through its arguments, etc., it is nevertheless a static entity. A def's UGen graph function (and the SC code within it) is evaluated only when the def is created. Thus statements like while, do, collect etc. will have no further effect at the time the def is used to create a Synth, and it is important to understand that a UGen graph function should not be designed in the same way as functions in the language, where multiple evaluations can yield different results. It will be evaluated once and only once.

NOTE: `if` is implemented as a linear signal crossfade when the receiver is an UGen

There are other ways of achieving similar results, however, often using UGens such as Rand. For example, the following def will have a single randomly generated frequency, which will be the same for every Synth based on it:

```
(
SynthDef(\help_notRand, { |out|
    Out.ar(out,
        SinOsc.ar(rrand(400, 800), 0, 0.2) * Line.kr(1, 0, 1, doneAction:
Done.freeSelf)
    )
}).add;
)
a = Synth(\help_notRand);
b = Synth(\help_notRand); // the same freq as a
```

This one on the other hand will have a different random freq for each Synth created:

```
(
SynthDef(\help_isRand, { |out|
    Out.ar(out,
        SinOsc.ar(Rand(400, 800), 0, 0.2) * Line.kr(1, 0, 1, doneAction:
Done.freeSelf)
    )
}).add;
)
a = Synth(\help_isRand);
b = Synth(\help_isRand); // a different randomly selected freq
```

Class Methods

`SynthDef.new(name, ugenGraphFunc, rates, prependArgs, variants, metadata)`

Create a SynthDef instance, evaluate the ugenGraphFunc and build the ugenGraph.

Arguments:

name	A String or Symbol (i.e. "name" or \name). This name will be used to refer to the SynthDef when creating a Synth based upon it, and should be unique.
ugenGraphFunc	An instance of Function specifying how the def's UGens are interconnected. See the discussion above for information on how the Function's arguments are specified.
rates	An optional Array of specifications for the ugenGraphFunc's arguments. The order corresponds to the order of arguments. See the examples below to see how these are used. A specification can be: **nil/zero**

\ar

An audio rate AudioControl is created.

a float

the Control will have a lag of the specified time. This can be used to create smooth transitions between different values. t_ and i_ args cannot be lagged.

\ir

The Control can be set only at creation ('initial rate'). See discussion above.

\tr

The Control is used as a trigger. See discussion above.

prependArgs An optional Array of objects which will be passed as the first arguments to the ugenGraphFunc when it is evaluated. Arguments which receive values in this way will not be converted to instances of Control. See the `wrap` example below for an example of how this can be used.

variants An optional Event containing default argument settings. These can override the defaults specified in the ugenGraphFunc. When creating a Synth a variant can be requested by appending the defName argument in the form 'name.variant' or "name.variant". See example below.

metadata An optional Event containing additional, user-defined information that is relevant to the use of the SynthDef in the client. The SynthDef itself is sent to the server for audio rendering; metadata are strictly client-side descriptive information. Currently the 'specs' key in the event is reserved for ControlSpecs to be associated with SynthDef arguments (this is useful for automatic GUI construction). Metadata can be persisted to disk and loaded automatically as part of a SynthDesc. See the SynthDesc help file for more details.

SynthDef.**wrap**(func, rates, prependArgs)

Wraps a function within an enclosing synthdef.

Discussion:

Arguments to the wrapped function are automatically promoted to be SynthDef controls, using the same rules applied to arguments of the main UGen function. For a very simple example:

```
d = SynthDef(\demoWrapping, { |out|
    Out.ar(out, SynthDef.wrap({ |freq| SinOsc.ar(freq) }))
});

d.allControlNames;
```

Prints: `[ControlName P 0 out control 0, ControlName P 1 freq control 0]`.

The outer function declares the argument 'out', and the wrapped function has 'freq' as its argument. The resulting SynthDef has both arguments as controls, exactly as if the outer function included both as arguments.

The rates array behaves as described earlier. `PrependArgs` allows values or unit generators to be passed into the inner function from the enclosing SynthDef context. Any inner function argument that receives a prependArg value (including nil) will use that value, suppressing creation of a control for that argument. The longer example below demonstrates this technique.

This is very useful for mass-producing SynthDefs that have a common "shell" defining features such as enveloping or triggering mechanisms that apply to different subgraphs of unit generators. The common features need be written only once; the UGens that differ between the SynthDefs are plugged into the supporting architecture.

SynthDef.**synthDefDir**

SynthDef.**synthDefDir** = dir

Get or set the default directory to which defs are written.

Remove the synthdef name from the SynthDescLib named libname and from its servers.

SynthDef.**writeOnce**(name, func, rates, prependArgs, variants, dir, metadata, mdPlugin)

Create a new SynthDef. It is written to disk only if a def file with this name does not already exist. Note that this will not check for differences, so you will need to delete the defFile to get it to rebuild. Default for dir is to use the path specified by SynthDef.synthDefDir.

WARNING: SynthDef.writeOnce is a legacy method. Its main use was to improve the efficiency of SynthDefs in quarks, but this is superseded by SynthDescLib. Being completely impervious to changes, it can cause difficult-to-diagnose bugs (such as having version 1.1 of a quark but with a SynthDef stuck in version 1.0). Quark developers should now use -add instead.

The exception is very large SynthDefs, where you have a choice between -writeDefFile and this method. Even then, the efficiency savings of writeOnce are only in disk I/O -- both methods build the SynthDef every time they run.

SynthDef.**warnAboutLargeSynthDefs**

SynthDef.**warnAboutLargeSynthDefs** = value

When a SynthDef is too large to be sent to a local server in a standard OSC message, it is written to disk and the server loads it from there instead. If warnAboutLargeSynthDefs is set to true, it will warn that this is happening. There will always be a warning for remote servers.

Inherited class methods

7 methods from Object ► show

Instance Methods

.add(libname, completionMsg, keepDef: true)

Adds the synthdef to the SynthDescLib specified by libname, and sends it to the library's servers. No defFile is written; all operations take place in memory.

Discussion:

After using this method, the synthdef can be used with event streams as in store(), but without the permanent artifact of a file on disk. Calling this method triggers an update message with the key \synthDescAdded for any dependants the library may have. This can be used to trigger additional behaviour every time a def/desc is added. See Object: Dependancy.

A server can be added by SynthDescLib.global.addServer(server).

Note that the "dir" and "mdPlugin" arguments do not exist for this method. Because no file is written, there is no need to specify a directory or write a metadata file.

```
(
SynthDef(\help_synth, { |out, freq = 800, sustain = 1, amp = 0.1|
    Out.ar(out,
        SinOsc.ar(freq, 0, 0.2) * Line.kr(amp, 0, sustain, doneAction:
Done.freeSelf)
    )
}).add;
)
```

.name

.name = value

Return this def's name.

.func = value

Return this def's ugenGraphFunc.

.variants

.variants = value

Return an Event containing this def's variants.

.allControlNames

.allControlNames = value

An array of ControlName's for the controls.

.specs

A Dictionary of ControlSpecs for a SynthDef. Equivalent to: `d.metadata[\specs]`

Special purpose methods

(for most purposes, the method add is recommended)

.writeDefFile(dir, overwrite: true, mdPlugin)

Writes the def as a file called name.scsyndef in a form readable by a server. Default for dir is synthdefs/. Defs stored in the default directory will be automatically loaded by the local and internal Servers when they are booted.

.load(server, completionMsg, dir)

Write the defFile and send a message to server to load this file. When this asynchronous command is completed, the completionMessage (a valid OSC message) is immediately executed by the server. Default for dir is synthdefs/.

.send(server, completionMsg)

Compile the def and send it to server without writing to disk (thus avoiding that annoying SynthDef buildup). When this asynchronous command is completed, the completionMessage (a valid OSC message) is immediately executed by the server.

.store(libname: 'global', dir, completionMsg, mdPlugin)

Write the defFile and store it in the SynthDescLib specified by libname, and send a message to the library's server to load this file. When this asynchronous command is completed, the completionMessage (a valid OSC message) is immediately executed by the server. Default for libname is \global, for dir is synthdefs/. This is needed to use defs with the event stream system. See Streams and Pattern.

Arguments:

libname	name of the SynthDescLib
dir	
completionMsg	
mdPlugin	(optional) The metadata plug-in class that will be used to persist metadata. If not supplied, the default plug-in is used. See the SynthDesc help file for details.

.play(target, args, addAction: 'addToHead')

list of valid addActions see Synth. The default is \addToHead.

Returns:

a corresponding Synth object.

Inherited instance methods

268 methods from Object ► show

Undocumented instance methods

.addAr(name, values)

.addConstant(value)

.addControlName(cn)

.addControlsFromArgsOfFunc(func, rates, skipArgs: 0)

.addCopiesIfNeeded

.addIr(name, values)

.addKr(name, values, lags)

.addNonControl(name, values)

.addTr(name, values)

.addUGen(ugen)

.asBytes

.asDefName
From extension in /usr/local/share/SuperCollider/SCClassLibrary/Common/Control/asDefName.sc

.asSynthDef
From extension in /usr/local/share/SuperCollider/SCClassLibrary/Common/Control/asDefName.sc

.asSynthDesc(libname: 'global', keepDef: true)

.available

.available = value

.build(ugenGraphFunc, rates, prependArgs)

.buildControls

.buildForProxy
From extension in /usr/local/share/SuperCollider/SCClassLibrary/JITLib/ProxySpace/wrapForNodeProxy.sc

.buildUgenGraph(func, rates, prependArgs)

.canFreeSynth
From extension in /usr/local/share/SuperCollider/SCClassLibrary/Common/Audio/canFreeSynth.sc

.canReleaseSynth
From extension in /usr/local/share/SuperCollider/SCClassLibrary/Common/Audio/canFreeSynth.sc

.children = value

.cleanupTopoSort

.collectConstants

.constantSet

.constantSet = value

.constants

.constants = value

.controlIndex

.controlIndex = value

.controlNames

.controlNames = value

.controls

.controls = value

.desc

.desc = value

.doSend(server, completionMsg)

.dumpUGens

.findSpecFor(controlName)

.finishBuild

.hasGateControl
From extension in /usr/local/share/SuperCollider/SCClassLibrary/Common/Audio/canFreeSynth.sc

.indexUGens

.initBuild

.initTopoSort

.loadReconstructed(server, completionMsg)

.maxLocalBufs

.maxLocalBufs = value

.metadata

.metadata = value

.numChannels
From extension in /usr/local/share/SuperCollider/SCClassLibrary/JITLib/ProxySpace/wrapForNodeProxy.sc

.optimizeGraph

.removeUGen(ugen)

.replaceUGen(a, b)

.setControlNames(controlUGens, cn)

.storeOnce(libname: 'global', dir, completionMsg, mdPlugin)

.topologicalSort

.widthFirstUGens

.widthFirstUGens = value

.writeConstants(file)

.writeDef(file)

.writeOnce(dir, mdPlugin)

Examples

Basic

```
// Note that constructions like SynthDef(...) and Synth(...) are short for
SynthDef.new(...), etc.
// With SynthDef it is common to chain this with calls on the resulting instance,
// e.g. SynthDef(...).add or SynthDef(...).play

// make a simple def and send it to the server

s.boot;
SynthDef(\SimpleSine, { |freq = 440, out| Out.ar(out, SinOsc.ar(freq, 0, 0.2))
}).add;

// the above is essentially the same as the following:
d = SynthDef(\SimpleSine, { |freq = 440, out| Out.ar(out, SinOsc.ar(freq, 0, 0.2))
});
d.add;

// now make a synth from it, using the default value for freq, then another with a
different value
x = Synth(\SimpleSine);
y = Synth(\SimpleSine, [\freq, 660]);

// now change the freq value for x
x.set(\freq, 880);

x.free; y.free;

// using the play convenience method
x = SynthDef(\SimpleSine, { |freq = 440, out| Out.ar(out, SinOsc.ar(freq, 0, 0.2))
}).play
x.free;
```

Argument Rates

```
(
```

```
    Out.ar(out, SinOsc.ar(freq+[0, 1], 0, Decay2.kr(t_trig, 0.005, 1.0)));
}, [0, 0, 4]        // lag the freq by 4 seconds (the second arg), but not t_trig
(won't work anyway)
).add;
)

y = Synth(\trigTest1);
y.set(\t_trig, 1); // you can do this multiple times
y.set(\t_trig, 1, \freq, 220); // hear how the freq lags
y.free;

// Pbind with the default \note event type, and Pmono, use an automatically-
generated
// function to look up argument values. This function translates \trig in the
// event to \t_trig in the synth.
(
Pmono(\trigTest1, *[
    trig: Prand([0, 1], inf), // for random trigger, use trig, not t_trig
    freq: Pwhite(lo: 220, hi: 440, length: 10),
    dur: 1
]).play;
)

// Note that With event type \set, there is no automatic translation so you
// should use t_trig in that case.
y = Synth(\trigTest1);
(
Pbind(*[
    type: \set,
    id: y.nodeID,
  args: #[\t_trig],
    t_trig: Prand([0, 1], 8)
]).play;
)
y.free;

// This second version makes trig a \tr arg by specifying it in the rates array.
(
SynthDef(\trigTest2, { |out, trig = 0, freq = 440|
    Out.ar(out, SinOsc.ar(freq+[0, 1], 0, Decay2.kr(trig, 0.005, 1.0)));
    }, [0, \tr, 4]        // lag the freq (lagtime: 4s), \tr creates a TrigControl
for trig
).add;
)

// There is a different way of writing the same thing using NamedControls
(
SynthDef(\trigTest2, { |out|
    Out.ar(out, SinOsc.ar(\freq.kr(440, 4) + [0, 1], 0, Decay2.kr(\trig.tr, 0.005,
1.0)));
}).add;
)

// Using the second version, create a synth
z = Synth(\trigTest2);

// Now trigger the decay envelope
z.set(\trig, 1);                // you can do this multiple times
z.set(\trig, 1, \freq, 220);    // hear how the freq lags
z.set(\trig, 1, \freq, 880);
z.free; // free the synth

// Use trig in Pbind and Pmono
(
Pmono(\trigTest2, *[
    trig: Prand([0, 1], inf), // for random trigger, use trig, not t_trig
    freq: Pwhite(lo: 220, hi: 440, length: 10),
    dur: 1
```

Variants

```
// create a def with some variants
(
SynthDef(\vartest, { |out, freq = 440, amp = 0.2, a = 0.01, r = 1|
    // the EnvGen with doneAction: Done.freeSelf frees the synth automatically
when done
    Out.ar(out, SinOsc.ar(freq, 0, EnvGen.kr(Env.perc(a, r, amp), doneAction:
Done.freeSelf)));
}, variants: (alpha: [a: 0.5, r: 0.5], beta: [a: 3, r: 0.01], gamma: [a: 0.01, r:
4])
).add;
)

// now make some synths. First using the arg defaults
Synth(\vartest);

// now the variant defaults
Synth('vartest.alpha');
Synth('vartest.beta');
Synth('vartest.gamma');

// override a variant
Synth('vartest.alpha', [\release, 3, \freq, 660]);
```

Array Arguments

```
// freqs is defined as an arrayed control. This makes a multichannel Control of
the same size.
(
SynthDef(\arrayarg, { |out, amp = 0.1, gate = 1|
    var env, sines;
    var freqs = NamedControl.kr(\freqs, [300, 400, 500, 600]);
    env = Linen.kr(gate, 0.1, 1, 1, 2) * amp;
    sines = SinOsc.ar(freqs +.t [0, 0.5]).cubed.sum; // A mix of 4 oscillators
    Out.ar(out, sines * env);
}, [0, 0.1, 0]).add;
)

x = Synth(\arrayarg);
x.setn(\freqs, [440, 441, 442, 443]);

// Values outside of the array bounds are ignored --
// 0 is dropped
x.setn(\freqs, [300, 400, 500, 600, 0]);

x.release;

// Mr. McCartney's more complex example
(
fork {
    z = Synth(\arrayarg);

    2.wait;
    10.do {
        z.setn(\freqs, { exprand(200, 800.0) } ! 4);
        (2 ** (0..3).choose * 0.2).wait;
    };

    z.set(\amp, -40.dbamp);

    10.do {
```

```
    };
    2.wait;

    z.release;
};
)
```

Wrapping Example: 'factory' production of effects defs

```
// The makeEffect function below wraps a simpler function within itself and
provides
// a crossfade into the effect (so you can add it without clicks), control over
wet
// and dry mix, etc.
// Such functionality is useful for a variety of effects, and SynthDef-wrap
// lets you reuse the common code.
(
// the basic wrapper
~makeEffect = { |name, func, lags, numChannels = 2|

    SynthDef(name, { |i_bus = 0, gate = 1, wet = 1|
        var in, sound, env, lfo;
        in = In.ar(i_bus, numChannels);
        env = Linen.kr(gate, 2, 1, 2, 2); // fade in the effect

        // call the wrapped function. The in and env arguments are passed to the
function
        // as the first two arguments (prependArgs).
        // Any other arguments of the wrapped function will be Controls.
        sound = SynthDef.wrap(func, lags, [in, env]);

        XOut.ar(i_bus, wet * env, sound);
    }, [0, 0, 0.1]).add;

};
)

// now make a wah
(
~makeEffect.value(\wah, { |in, env, rate = 0.7, ffreq = 1200, depth = 0.8, rq =
0.1|
    // in and env come from the wrapper. The rest are controls
     var lfo;
    lfo = LFNoise1.kr(rate, depth * ffreq, ffreq);
    RLPF.ar(in, lfo, rq, 10).distort * 0.15;
},
[0.1, 0.1, 0.1, 0.1], // lags for rate ffreq, depth and rq
2    // numChannels
);
)

// now make a simple reverb
(
~makeEffect.value(\reverb, { |in, env|
    // in and env come from the wrapper.
    var input;
    input = in;
    16.do({ input = AllpassC.ar(input, 0.04, Rand(0.001, 0.04), 3) });
    input
},
nil, // no lags
2    // numChannels
);
)

// something to process
```

```
y = Synth.tail(s, \wah);
z = Synth.tail(s, \reverb, [\wet, 0.5]);

// we used an arg named gate, so Node-release can crossfade out the effects
y.release;

// setting gate to zero has the same result
z.set(\gate, 0);

x.free;
```

common argument names: out and gate

```
// arguments named 'out' and 'gate' are commonly used to specify an output bus and
// EnvGen gate respectively. Although not required, using them can help with
consistency
// and interchangeability. 'gate' is particularly useful, as it allows for Node's
release
// method.
(
SynthDef(\synthDefTest, { |out, gate = 1, freq = 440|
    // doneAction: Done.freeSelf frees the synth when EnvGen is done
    Out.ar(out, SinOsc.ar(freq) * EnvGen.kr(Env.asr(0.1, 0.3, 1.3), gate,
doneAction: Done.freeSelf));
}).store; // use store for compatibility with pattern example below
)

x = Synth(\synthDefTest, [\out, 0]); // play out through hardware output bus 0
(see Out.help)
x.release; // releases and frees the synth (if doneAction is > 2; see EnvGen)

// equivalent:

x = Synth(\synthDefTest); // out defaults to zero, if no default arg is given.
x.set(\gate, 0);

// if value is negative, it overrides the release time, to -1 - gate
x = Synth(\synthDefTest);
x.set(\gate, -5); // 4 second release

// equivalent:
x = Synth(\synthDefTest);
x.release(4);

// if the out arg is used in a standard way, it can always be changed without
knowing the synth def
x = Synth(\synthDefTest, [\out, 0]);
x.set(\out, 1); // play through channel 1
x.release;

// Another good example of this is with patterns, which can use gate to release
notes
(
Pbind(
    \instrument, \synthDefTest,
    \freq, Pseq([500, 600, Prand([200, 456, 345], 1)], inf),
    \legato, Pseq([1.5, 0.2], inf),
    \dur, 0.4,
    \out, Pseq([0, 1], inf)
).play;
)
```

Inline ControlSpec definitions

```
(
~def = SynthDef(\argumentExample, { |freq, amp|
    var sig;
    freq.spec = ControlSpec(20, 20000, \exp);
    amp.spec = ControlSpec(0, 1);
    sig = SinOsc.ar(freq, 0, amp);
    Out.ar(0, sig);
});
~def.specs.freq.postln;
~def.specs.amp.postln;
)

(
~def = SynthDef(\argumentExample, {
    var sig;
    sig = SinOsc.ar(
        \freq.kr(spec: ControlSpec(20, 20000)),
        0,
        \amp.kr(spec: ControlSpec(0, 1));
    );
    Out.ar(0, sig);
});
~def.specs.freq.postln;
~def.specs.amp.postln;
)
```

helpfile source: /usr/local/share/SuperCollider/HelpSource/Classes/SynthDef.schelp
link::Classes/SynthDef::